CyberChessCoach

QA Test Plan

Version 1.0 | March 2026

AI-powered chess training system with strict component isolation

1. Purpose & Scope

This test plan defines the QA strategy, test areas, and individual test cases for the CyberChessCoach system. It is intended as a living document to use as both an onboarding reference and an active test registry.

CyberChessCoach is a multi-component mono-repo comprising:

- Android App — UI, board rendering, game orchestration
- C++ Chess Engine — ~1800 Elo opponent move generation
- LLM Explanation Engine — RAG-powered, Stockfish-backed, safety-validated explanations

The system enforces three non-negotiable architectural invariants that must be continuously validated:

- The opponent engine NEVER explains moves
- Stockfish NEVER plays moves
- The LLM NEVER calculates or suggests moves

1.1 What Is Already Automated

The following test categories already exist in code and are run via CI/CD (GitHub Actions, 489+ runs):

- Golden tests — lock ESV, RAG, and prompt snapshot behavior
- Negative tests — forbid illegal LLM explanations
- Regression tests — ensure explanation quality
- Validator tests — fake LLM simulates violation scenarios
- Contract tests — prove all SECA safety rules are enforced

This QA plan extends these with manual, exploratory, integration, and end-to-end test cases that complement the automated suite.

2. Test Area Overview

Area	Scope	Risk Level
Architectural Invariants	Verify the 3 core invariants are never violated across all paths	**Critical**
LLM Output Validation	Safety contracts, validator checks, edge case explanations	**Critical**
C++ Chess Engine	Move legality, Elo strength, edge positions, draw/mate detection	**High**
Android Application	Board UI, move input, explanation display, error handling	**High**
Stockfish Integration	Evaluation accuracy, JSON output format, ESV signal extraction	**High**
RAG Pipeline	Document retrieval relevance, determinism, empty-result handling	**High**
API & Backend	Health check, endpoints, auth, error responses	**Medium**
Docker & Deployment	Container build, env vars, Fly.io / Hetzner deploy health	**Medium**
Hetzner Production Server	Bootstrap, DB, Ollama, Trivy scans, network security, E2E on live infra	**High**
CI/CD Pipeline	Workflow triggers, gate enforcement, dependabot PRs	**Medium**
Security	Scheduled security scan, secret exposure, dependency vulnerabilities	**Medium**

3. Architectural Invariant Tests

These are the highest-priority test cases in the entire plan. A failure here constitutes a P0 blocker.

ID	Area	Test Case	Priority	Type	Expected Result
INV-01	Invariants	Verify LLM output never contains a suggested move in algebraic notation (e.g. e4, Nf3, O-O)	**P0**	Automated / Contract	No move notation found in any LLM explanation response
INV-02	Invariants	Verify C++ engine response never contains explanatory text — only a move token	**P0**	Automated / Unit	Engine output is exactly one move string, no additional text

ID	Area	Test Case	Priority	Type	Expected Result
INV-03	Invariants	Verify Stockfish is never called during the opponent move generation phase	**P0**	Automated / Contract	No Stockfish invocation in the engine call path
INV-04	Invariants	Verify LLM is never called before both moves are committed and board is frozen	**P0**	Integration	LLM call only occurs after step 4 (board update) in the data flow
INV-05	Invariants	Verify no component depends on LLM output for any game decision	**P0**	Contract / Review	Game logic has zero dependency on explanation layer output

4. LLM Output Validation Tests

The LLM is explicitly untrusted. All output must pass the validator pipeline before reaching the user.

ID	Area	Test Case	Priority	Type	Expected Result
LLM-01	LLM Validator	Explanation passes when valid: no move suggestions, no engine mentions, correct mate handling	**P0**	Automated / Golden	Validator returns PASS, explanation delivered to Android app
LLM-02	LLM Validator	Explanation blocked when it contains an engine mention (e.g. 'Stockfish says...')	**P0**	Automated / Negative	Validator returns FAIL, fallback response returned, error logged
LLM-03	LLM Validator	Explanation blocked when it contains invented tactical moves not found in RAG context	**P0**	Automated / Negative	Validator returns FAIL, fallback triggered
LLM-04	LLM Validator	Forced mate position: explanation correctly identifies mate without inventing lines	**P0**	Automated / Regression	Explanation references mate, no suggested continuation moves
LLM-05	LLM Validator	Missing ESV data: LLM issues explicit refusal rather than hallucinating	**P1**	Automated / Contract	Explicit refusal phrase present in output, no fabricated evaluation
LLM-06	LLM Validator	Explanation does not reproduce raw centipawn scores from Stockfish JSON	**P1**	Automated / Golden	No numeric score values appear verbatim in explanation text

ID	Area	Test Case	Priority	Type	Expected Result
LLM-07	LLM Output	Explanation is coherent and references the correct side (White/Black) relative to user	**P2**	Manual / Exploratory	Explanation matches the game position and player perspective

5. C++ Chess Engine Tests

ID	Area	Test Case	Priority	Type	Expected Result
ENG-01	Engine	Engine returns a legal move for all standard opening positions	**P0**	Automated / Unit	Move is valid per chess rules, no illegal moves accepted
ENG-02	Engine	Engine detects and plays checkmate when available in 1 move	**P0**	Automated / Unit	Engine plays the mating move, game terminates correctly
ENG-03	Engine	Engine handles stalemate position — returns no move, signals draw	**P0**	Automated / Unit	Stalemate detected, game ends as draw
ENG-04	Engine	Engine handles promotion positions — correctly promotes pawn	**P1**	Automated / Unit	Promoted piece returned correctly (default Queen)
ENG-05	Engine	Engine performance: responds within acceptable time limit (e.g. 5s) for all positions	**P1**	Performance	No response exceeds the timeout threshold
ENG-06	Engine	Engine operates with no knowledge of Stockfish evaluations or LLM output	**P0**	Contract / Review	Engine module has no imports or calls to Stockfish or LLM layer

6. Android Application Tests

ID	Area	Test Case	Priority	Type	Expected Result
AND-01	Android UI	Board renders all 64 squares with correct initial piece positions	**P1**	Manual / UI	Board matches standard chess starting position
AND-02	Android UI	User can input a legal move; board updates to reflect it	**P0**	Manual / E2E	Move appears on board; opponent move follows

ID	Area	Test Case	Priority	Type	Expected Result
AND-03	Android UI	User inputs an illegal move; error message displayed, board unchanged	**P0**	Manual / E2E	Error shown, board not updated, input re-accepted
AND-04	Android UI	Explanation panel displays text after each move pair is completed	**P1**	Manual / E2E	Non-empty explanation appears within acceptable time
AND-05	Android UI	Logout flow: user session cleared, board reset, returns to login screen	**P1**	Manual / UI	No residual game state after logout
AND-06	Android UI	Game summary screen shows move quality scores at game end	**P1**	Manual / UI	Summary screen displays per-move quality data
AND-07	Android	Android app handles LLM timeout gracefully — shows fallback message	**P1**	Manual / E2E	Fallback message displayed, game remains playable
AND-08	Android	App does not crash on network loss mid-game	**P1**	Manual / Resilience	App shows offline state, no crash, resumes on reconnect

7. Stockfish & ESV Pipeline Tests

ID	Area	Test Case	Priority	Type	Expected Result
SF-01	Stockfish	Stockfish returns valid JSON with centipawn_loss, is_winning, is_forced_mate fields	**P0**	Automated / Unit	JSON schema validated, all required fields present
SF-02	Stockfish	ESV correctly coarsens raw centipawn scores — no raw scores in signal vector	**P0**	Automated / Golden	ESV output contains no numeric centipawn values
SF-03	Stockfish	ESV rejects malformed Stockfish JSON — returns error, does not propagate	**P1**	Automated / Negative	ESV raises validation error, pipeline halts cleanly
SF-04	Stockfish	ESV signal is deterministic: same FEN input always produces identical output	**P0**	Automated / Regression	100 runs with same FEN produce identical ESV output
SF-05	Stockfish	setup_stockfish.py correctly downloads and configures Stockfish binary	**P1**	Manual / Setup	Binary present and executable after setup script runs

8. RAG Pipeline Tests

ID	Area	Test Case	Priority	Type	Expected Result
RAG-01	RAG	RAG retrieval is deterministic: same ESV always returns same documents	**P0**	Automated / Golden	Document set identical across repeated calls with same input
RAG-02	RAG	RAG returns relevant strategic documents for a winning position ESV	**P1**	Automated / Regression	Retrieved documents contain positional/strategic principles
RAG-03	RAG	RAG handles empty document store gracefully — no crash, signals missing data	**P1**	Automated / Negative	Explicit error or empty signal returned; no exception propagation
RAG-04	RAG	RAG documents are never modified by the LLM or any downstream component	**P0**	Contract / Review	Document store is read-only at runtime

9. API & Backend Tests

ID	Area	Test Case	Priority	Type	Expected Result
API-01	Backend	GET /health returns 200 OK with valid response body	**P0**	Automated / Smoke	200 response, expected JSON body
API-02	Backend	POST /inference with valid FEN returns explanation within SLA	**P0**	Automated / Integration	200 response with explanation text, no invariant violations
API-03	Backend	POST /inference with invalid FEN returns 400 with descriptive error	**P1**	Automated / Negative	400 response, no stack trace exposed, error message present
API-04	Backend	Engine auth is required — unauthenticated engine calls return 401	**P1**	Automated / Security	401 returned for missing/invalid auth token
API-05	Backend	Exponential backoff fires on repeated LLM failures — does not flood provider	**P1**	Automated / Resilience	Delay between retries increases, max retries not exceeded
API-06	Backend	seca_doctor.py reports all safety checks as PASS on clean install	**P0**	Manual / Smoke	All checks green, no FAIL or WARN items

10. Docker & Deployment Tests

ID	Area	Test Case	Priority	Type	Expected Result
DEP-01	Docker	docker-compose up builds and starts all services without errors	**P1**	Manual / Smoke	All containers healthy, no startup errors
DEP-02	Docker	seca.db is NOT baked into the Docker image (should be volume-mounted)	**P1**	Manual / Review	Image layer does not contain seca.db
DEP-03	Docker	Environment variables (API keys, secrets) are not hardcoded in Dockerfile	**P0**	Security / Review	No secrets present in image layers or compose file
DEP-04	Deployment	Hetzner bootstrap script completes without errors on fresh host	**P1**	Manual / Deployment	Server bootstrapped, service reachable on expected port
DEP-05	Deployment	Production env template: all required variables documented and present	**P1**	Manual / Review	No missing env vars on production deploy
DEP-06	Docker	verify_safety.py passes inside the container (not just locally)	**P0**	Automated / CI	All SECA safety checks pass in containerized environment

11. Hetzner Production Server Tests

The project migrated from Fly.io to a Hetzner VPS hosting the full production stack: Python backend, Postgres DB, and a self-hosted Ollama LLM instance. This section covers testing the Hetzner environment end-to-end.

11.1 Prerequisites & Access

Before running any Hetzner tests, confirm the following with the project owner:

- SSH access to the Hetzner VPS (key-based, not password)
- Production .env file obtained securely (never committed to git)
- Hetzner server IP / hostname and port
- Postgres connection string (host, port, db name, credentials)
- Ollama endpoint URL and model name in use
- seca_doctor.py run result from the server (screenshot or log)

11.2 Bootstrap & Initial Setup Tests

ID	Area	Test Case	Priority	Type	Expected Result
HTZ-01	Hetzner / Bootstrap	Run bootstrap script on a clean Hetzner instance — completes without errors	**P0**	Manual / Deployment	Script exits 0, all services start, no missing dependency errors
HTZ-02	Hetzner / Bootstrap	All required env vars are present after bootstrap (no undefined vars on startup)	**P0**	Manual / Smoke	Application starts without KeyError or missing config exceptions
HTZ-03	Hetzner / Bootstrap	Postgres DB is created and reachable from the app container	**P0**	Manual / Smoke	DB connection succeeds; app logs show no connection refused errors
HTZ-04	Hetzner / Bootstrap	Stockfish binary is installed and executable inside the container	**P0**	Manual / Smoke	setup_stockfish.py exits 0; stockfish --version returns a version string
HTZ-05	Hetzner / Bootstrap	Ollama is running and the configured model is loaded and responding	**P0**	Manual / Smoke	curl to Ollama endpoint returns a valid model response

11.3 Service Health & Connectivity Tests

ID	Area	Test Case	Priority	Type	Expected Result
HTZ-06	Hetzner / Health	GET /health on the Hetzner server returns 200 OK	**P0**	Manual / Smoke	curl http://<hetzner-ip>:<port>/health → 200 with expected body
HTZ-07	Hetzner / Health	seca_doctor.py reports all PASS on the live server (not just locally)	**P0**	Manual / Smoke	All safety invariant checks green on production host
HTZ-08	Hetzner / Health	verify_safety.py passes on the live server	**P0**	Manual / Smoke	No FAIL or WARN output from safety verification script
HTZ-09	Hetzner / Network	App container can reach Ollama container on internal network (not public internet)	**P1**	Manual / Network	LLM call succeeds; Ollama not exposed on public port
HTZ-10	Hetzner / Network	Postgres port is NOT exposed to the public internet	**P0**	Security / Network	nmap or Hetzner firewall rules confirm port 5432 is closed externally
HTZ-11	Hetzner / Network	Only necessary ports are open (e.g. 80/443 + app port) — all others closed	**P1**	Security / Network	Hetzner firewall config matches expected minimal surface

11.4 End-to-End Flow on Hetzner

ID	Area	Test Case	Priority	Type	Expected Result
HTZ-12	Hetzner / E2E	POST /inference with a valid FEN returns a passing explanation from Ollama	**P0**	Manual / E2E	200 response, explanation text present, no validator failures in logs
HTZ-13	Hetzner / E2E	Explanation pipeline completes within acceptable latency on Hetzner hardware	**P1**	Manual / Performance	Full inference round-trip (FEN → explanation) completes under 30s
HTZ-14	Hetzner / E2E	Stockfish evaluation step runs correctly on the server CPU	**P0**	Manual / E2E	ESV output logged correctly; no Stockfish crash or timeout
HTZ-15	Hetzner / E2E	RAG document retrieval works correctly with the production document store	**P1**	Manual / E2E	Documents retrieved match expected set for a standard position ESV
HTZ-16	Hetzner / E2E	Android app connects to Hetzner backend and completes a full game with explanations	**P0**	Manual / E2E	Game plays to completion; all explanations displayed; no crashes

11.5 Persistence & Database Tests

ID	Area	Test Case	Priority	Type	Expected Result
HTZ-17	Hetzner / DB	Game history is persisted to Postgres after a completed game	**P1**	Manual / DB	SELECT from game history table returns the completed game record
HTZ-18	Hetzner / DB	DB survives container restart — data is not lost on redeploy	**P1**	Manual / Resilience	Game records present after docker-compose down/up cycle
HTZ-19	Hetzner / DB	seca.db (SQLite) is volume-mounted, not inside the container image	**P0**	Manual / Review	docker inspect confirms seca.db is a bind mount, not baked in
HTZ-20	Hetzner / DB	DB credentials are sourced from env vars — not hardcoded in any config file	**P0**	Security / Review	grep for credentials in source returns no matches; env vars used throughout

11.6 Trivy Security Scan Tests

The repo uses Trivy for container image scanning (stabilised in the Mar 9 commits). These tests validate that the Hetzner image passes the scan gates.

ID	Area	Test Case	Priority	Type	Expected Result
HTZ-21	Hetzner / Security	Trivy image scan on the production Hetzner image returns no CRITICAL vulnerabilities	**P0**	Automated / Security	Trivy exits 0 or with no CRITICAL/HIGH findings above threshold
HTZ-22	Hetzner / Security	Trivy runtime manifest scan passes (hardened container runtime check)	**P1**	Automated / Security	No runtime manifest violations in scan output
HTZ-23	Hetzner / Security	No secrets or API keys detected by Trivy secret scanning in the image	**P0**	Automated / Security	Trivy secret scan returns 0 findings
HTZ-24	Hetzner / Security	Scheduled Security workflow targets the Hetzner image tag (not only local builds)	**P1**	Automated / CI	Workflow logs confirm Hetzner image pulled and scanned

11.7 Ollama (Self-hosted LLM) Tests

The project uses a VPS-hosted Ollama instance as the LLM provider on Hetzner, replacing any cloud LLM API.

ID	Area	Test Case	Priority	Type	Expected Result
HTZ-25	Hetzner / Ollama	Ollama responds to a direct prompt within acceptable latency	**P0**	Manual / Smoke	curl to Ollama API returns a completion within 20s
HTZ-26	Hetzner / Ollama	LLM output from Ollama still passes all SECA validators (same as cloud LLM)	**P0**	Manual / Contract	Validator pipeline passes; no invariant violations from Ollama output
HTZ-27	Hetzner / Ollama	App falls back gracefully if Ollama is temporarily unavailable (restart simulation)	**P1**	Manual / Resilience	Fallback explanation shown to user; exponential backoff fires; no crash
HTZ-28	Hetzner / Ollama	Ollama model version is pinned — not floating to latest automatically	**P1**	Manual / Review	Model tag in config is a fixed version, not 'latest'
HTZ-29	Hetzner / Ollama	Ollama is not accessible from the public internet (internal only)	**P0**	Security / Network	Port scan from external confirms Ollama port is not reachable

11.8 Hetzner Test Checklist (Quick Reference)

Use this checklist for each production deployment or smoke test session:

✓	Check	Result	Notes
☐	GET /health → 200 OK	Pass / Fail	
☐	seca_doctor.py all PASS	Pass / Fail	
☐	verify_safety.py passes	Pass / Fail	
☐	Ollama responding (curl test)	Pass / Fail	
☐	Postgres reachable from app	Pass / Fail	
☐	Postgres NOT exposed externally	Pass / Fail	
☐	Ollama NOT exposed externally	Pass / Fail	
☐	Trivy image scan: 0 CRITICAL	Pass / Fail	
☐	Full E2E game with explanation	Pass / Fail	
☐	No secrets in env or image	Pass / Fail	

12. CI/CD Pipeline Tests

ID	Area	Test Case	Priority	Type	Expected Result
CI-01	CI/CD	CI/CD workflow triggers on every push to main and every PR	**P1**	Automated / Meta	Workflow run appears within 60s of push/PR event
CI-02	CI/CD	Security workflow runs on schedule (daily/weekly) independently of pushes	**P1**	Automated / Meta	Scheduled run visible in Actions history at expected interval
CI-03	CI/CD	Dependabot PRs trigger both CI/CD and Security workflows	**P1**	Automated / Meta	Both workflow types appear on dependabot PR
CI-04	CI/CD	Black formatting check blocks merge if code is not formatted	**P1**	Automated / Gate	CI fails on unformatted code, merge blocked
CI-05	CI/CD	Android release build step fails fast on compilation error	**P1**	Automated / Gate	Build step fails with clear error, deploy step skipped
CI-06	CI/CD	Deploy step only runs after all test/build steps pass	**P0**	Automated / Gate	Deploy job has correct depends_on chain; never runs on test failure

13. Entry Criteria & Definition of Done

12.1 Entry Criteria (before QA begins a sprint)

- Feature branch merged to main with passing CI
- Docker image builds successfully
- verify_safety.py passes
- API /health endpoint reachable

12.2 Definition of Done (for a test case)

- Test case executed with documented actual result
- Pass/Fail status recorded
- Failing cases have a linked GitHub Issue
- P0 failures block release; P1 failures require sign-off

12.3 Release Gate

- All P0 test cases: PASS
- All architectural invariant tests (INV-01 through INV-05): PASS
- API-01 and API-06 smoke tests: PASS
- No open P0 issues

14. Priority & Type Legend

Priority	Meaning	Type	Meaning
P0	Blocker — release cannot ship	**Contract**	Enforces architectural invariant
P1	High — must fix before release	**Automated**	Runs in CI without human intervention
P2	Medium — fix in next sprint	**Manual**	Requires human execution
P3	Low — nice to have	**Exploratory**	No fixed script, tester uses judgment